

Claude Code 长上下文续航：Swap 与 Forge 实战教程

版本：2026-06-12 v3

这是一份面向外部读者的通用教程，讲的是如何在 Claude Code 这类 CLI Agent 长时间运行时，安全地“换窗续航”，避免上下文越积越重、会话卡死或恢复错位。

1. 这份教程解决什么问题

Claude Code 的长期会话通常会遇到三个问题：

- 对话越来越长，context 变重，回复变慢。
- 会话中混入过时的工具输出、错误提示或无关历史，后续推理被污染。
- 进程重启、tmux 恢复、watcher 接管时，可能接到错误 session。
- 过度裁剪后，新 session 虽然能看到工具 schema，却缺少最近的工具调用先例，导致工具幻觉或不会真正调工具。

最朴素的办法是“关掉重开”，但这会丢失工作现场。更好的做法是把必要上下文裁剪出来，接到一个新的干净 session。

这套思路可以分成两种机制：

机制	适合场景	核心动作
Swap	日常换窗、切模型、context 偏高但还没污染	快速裁剪近期 transcript，然后 resume
Forge	上下文明显污染、历史过肥、需要恢复到干净边界	深度过滤 transcript，重写链路后 resume

后面还会单独讲 Route Guard。它不是第三种换窗机制，而是一种“发现模型路由漂移后，触发 Forge 恢复”的保护策略。

2. 基本概念

Claude Code 的会话记录会落到本地 transcript 文件里，通常是 JSONL 格式：

```
~/claude/projects/<project-id>/<session-id>.jsonl
```

JSONL

的特点是：每一行都是一个完整事件。实现续航时，最重要的原则是按“行”裁剪，而不是按字节或字符串硬切。

安全裁剪要保留：

- 最近的 user/assistant 对话。
- 必要的工具调用结果摘要。
- 少量完整的工具调用回合，用来维持 agent 的工具使用能力。
- 当前任务所需的最小上下文。

安全裁剪要丢掉：

- 过早的历史对话。
- 大块无关工具输出。
- 临时调试噪音。
- 不适合跨 session 携带的内部推理块或损坏事件。

这里的“工具调用回合”不是一句文字说明，而是 transcript 里的真实结构：assistant 的 `tool_use` 事件，加上后续 user 侧匹配 `tool_result`

的事件。只写“我刚才调用了某工具”没有同样效果，反而可能让模型把工具调用当成普通文本模仿。

3. Swap：日常快速换窗

Swap 可以理解成“轻量续航”。

适合在这些时候触发：

- CLI 明显变慢。
- 想让新的模型参数、权限参数或配置生效。
- 当前对话还算干净，只是上下文太长。
- 需要让用户感觉“暂停一下，然后继续”。

推荐流程：

1. 确认当前没有 reply 正在 streaming。
2. 找到当前 active transcript。
3. 从尾部保留一段近期上下文。
4. 写成新的 session JSONL。
5. 用 `claude --resume <new-session-id>` 启动。
6. 发送一个验证 ping，确认新 transcript 会增长。
7. 记录 last-good session，供守护进程恢复。

Swap 的目标是快、稳、低打扰。它不一定要做很复杂的语义清洗，但必须避免切坏 JSONL。

4. Forge：深度重铸上下文

Forge 可以理解成“重铸一个干净 session”。

适合在这些时候触发：

- transcript 已经非常大。
- 上下文里出现持续影响后续回答的污染段。
- 普通 Swap 后仍然异常。
- 需要自动 watcher 在后台处理。

Forge 相比 Swap 多做几件事：

- 过滤不可携带事件。
- 找更干净的裁剪边界。

- 保留或补入少量完整工具调用回合，避免新 session 工具能力退化。
- 重写 session id 和 parent 链。
- 归档旧 transcript。
- 失败时清理半成品，避免守护进程误恢复。

Forge 的目标是把“最近必要上下文”整理成一个新的可信起点。

5. 架构总览

一个稳的续航系统通常由六部分组成：

1. Runtime：tmux 或类似 PTY 容器，用来承载 Claude Code CLI。
2. Transcript Reader：定位并读取当前 JSONL。
3. Trimmer/Rewriter：裁剪、过滤、重写 session 链。
4. Verifier：resume 后发 ping，检查 transcript 是否增长。
5. Guardian：进程挂掉时，只恢复 last-good session。
6. Route Guard：发现目标模型和实际模型不一致时，触发干净恢复。

推荐数据流：

```
current transcript
-> filter events
-> choose clean boundary
-> ensure tool primer
-> rewrite session id / parent chain
-> write new transcript
-> claude --resume <new-session-id>
-> verify transcript growth
-> mark last-good
```

6. 实现要点

6.1 定位 active transcript

不要只靠“最新修改时间”。更稳的方式是：

- 先从 tmux pane pid 找到 Claude Code 子进程。
- 再从本地 session metadata 反查 session id。
- 最后才 fallback 到 mtime。

mtime fallback 可以保留，但不要作为唯一来源。

6.2 按事件裁剪

错误做法：

```
读取整个文件 -> 截掉前 N 个字节 -> 写回
```

正确做法：

逐行读 JSONL -> 每行解析成 event -> 从尾部累计 token -> 保留完整 event

粗略 token 估算可以用 JSON 字符数除以 3 或

4。实际阈值需要根据模型上下文窗口、系统提示长度和工具定义大小调整。

6.3 重写链路

新 transcript 不只是复制旧内容，还要保证：

- 文件名是新的 session id。
- 事件里的 session id 对齐新 id。
- parent/uuid 链路连续。
- 不留下指向旧 session 的断裂引用。

如果链路不一致，resume 可能失败，或者失败得不明显。

6.4 验证 resume

只看到进程启动不代表 resume 成功。更可靠的验证是：

1. 记录新 transcript 初始大小。
2. 启动 `claude --resume <new-session-id>`。
3. 发送一条短 ping。
4. 检查新 transcript 是否增长。

这个验证能避免“进程在，但没有真正写入新会话”的假成功。

6.5 保留工具调用锚点

Forge 最容易被忽略的后遗症是：裁剪太干净时，模型可能在新 session 里短暂丢失工具使用惯性。

典型表现：

- 明明 MCP 或内置工具 schema 还在，模型却说自己不能用工具。
- 模型用自然语言“假装调用工具”，但 transcript 里没有真实 `tool_use`。
- Forge 后第一轮复杂任务明显比 Forge 前更容易幻觉。

解决方式是增加一个 Tool Primer，也就是“工具调用锚点”：

1. 正常裁剪后，检查保留内容里有没有完整工具回合。
2. 如果没有，向更早的 keepable events 里回捞最近 1 到 2 个成功工具回合。
3. 每个工具回合至少包含 assistant `tool_use` 和匹配的 user `tool_result`。
4. 工具结果要压缩，只保留成功/失败状态和很短摘要，避免把大块旧输出带回新 session。
5. 不保留内部推理块，不保留敏感输出，不把工具调用改写成自然语言。
6. 重写链路时，保持 `tool_use_id`、`sourceToolAssistantUUID` 等引用一致。

推荐阈值：

- 最少保留 1 个完整工具回合。
- 更稳的做法是保留 2 个不同类型的工具回合。

- Tool Primer 总预算控制在 2k 到 3k tokens 内。

Tool Primer 的目标不是保存工具结果，而是给新 session 一个真实、结构化的“工具使用先例”。

7. 模型路由漂移与 Route Guard

长期运行的 CLI Agent 还有一类隐蔽问题：用户配置了目标模型，但服务端实际执行时可能被路由到另一个模型、降级模型或临时实验分支。

这个问题麻烦在于：

- 模型自己未必知道自己被路由了。
- 普通日志里不一定第一眼能看出来。
- 被路由后的回复会写进 transcript。
- 如果直接 resume，这段历史还会继续影响后续上下文。

Route Guard 的目标是：发现“配置目标”和“实际响应模型”发生漂移时，自动从漂移前的干净边界 Forge 出一个新 session。

7.1 怎么识别

识别路由漂移至少需要两个信号：

1. 配置目标模型：来自配置文件、启动参数或用户显式选择。
2. 实际响应模型：来自 transcript 中 assistant event 的 model 字段，或等价的运行时元数据。

如果二者不一致，就可能发生了路由漂移。

但不要只看一次不一致就立刻恢复。更稳的判断条件是：

- 用户最近没有主动切模型。
- 配置目标在一段时间内保持稳定。
- 最新 assistant event 的实际模型偏离目标。
- 偏离发生在明确的时间点或连续多次出现。

这样可以避免把用户正常切模型误判成异常路由。

7.2 怎么恢复

Route Guard 的恢复流程可以复用 Forge：

1. 扫描 transcript，找到第一次模型漂移的位置。
2. 向前找一个干净锚点，避免切在工具调用中间。
3. 丢弃漂移点之后的污染段。
4. 保留漂移前的必要上下文。
5. 重写 session 链，生成新 JSONL。
6. 用目标模型参数重新 --resume。
7. 验证新 transcript 写入，并确认实际模型回到目标。

关键是：不要只“重新启动同一个上下文”。如果污染段已经写入 transcript，单纯重启会把污染继续带过去。

7.3 伪代码

```
def scan_route_guard(events, target_model):
    drift_line = None

    for index, event in enumerate(events):
        if event["type"] != "assistant":
            continue
        actual = event.get("message", {}).get("model")
        if actual and normalize(actual) != normalize(target_model):
            drift_line = index
            break

    if drift_line is None:
        return None

    anchor = find_clean_boundary_before(events, drift_line)
    return {
        "reason": "model_route_drift",
        "anchor": anchor,
        "drift_line": drift_line,
        "target_model": target_model,
    }
```

真实实现里还要加冷却、重试上限、误判保护和人工可观测日志。

7.4 验收标准

Route Guard 恢复后，至少确认：

- 新 transcript 不再包含漂移段。
- 新启动命令仍然带目标模型参数。
- resume 验证通过，transcript 会继续增长。
- 最新 assistant event 的实际模型回到目标模型。
- last-good 更新到恢复后的新 session。

8. 守护进程规则

守护进程是续航系统最容易踩坑的地方。

推荐规则：

- 只恢复 last-good session。
- 不要在找不到 last-good 时自动 --continue。
- Swap/Forge 运行时写一个短期 marker，让 guardian 暂停抢启。
- 续航成功后再更新 last-good。
- 续航失败要清理半成品 transcript。

最危险的失败模式是：新 session 还没验证成功，guardian 抢先启动了一个空 session，之后系统误以为恢复完成。

9. 触发策略

可以同时保留手动触发和自动触发。

手动触发适合：

- 用户感觉卡顿。
- 切模型或切配置。
- 维护者排障。

自动触发适合：

- transcript token 超过阈值。
- 文件持续增长到风险区。
- 检测到可恢复的异常输出段。
- 检测到模型路由漂移，且满足误判保护条件。

自动触发必须有冷却时间和互斥锁，避免连续触发多个恢复任务。

10. 失败回滚

一个成熟实现要准备三条路：

1. Forge/Swap 成功：标记 new session 为 last-good。
2. Forge/Swap 失败：删除半成品，继续使用 old session 或恢复 last-good。
3. old session 已经被杀：guardian 用 last-good resume。

不要把“刚写出来的新 JSONL”立刻当成可信 session。只有验证通过后，它才应该进入 last-good。

11. 验收清单

每次改续航逻辑，建议至少验收这些点：

- 能定位到正确 active transcript。
- 裁剪后 JSONL 每行都能解析。
- 新 session id 与文件名一致。
- parent 链路没有断。
- 至少保留 1 个完整工具调用回合，或明确记录为什么没有可保留工具回合。
- 保留的 tool_use 和 tool_result 能按 id 配对。
- claude --resume 能启动。
- ping 后新 transcript 增长。
- last-good 更新到新 session。
- guardian 不会抢启空 session。
- 失败时半成品会被清理。
- 用户侧能继续正常对话。

- 如果启用 Route Guard，目标模型和实际模型的对比结果可观测。
- Route Guard 不会把用户主动切模型误判成漂移。

12. 常见故障

resume 后没有输出

检查：

- 新 transcript 是否真的增长。
- ping 是否写入了新 session。
- tmux 里启动命令是否带了正确的 --resume。
- session id 是否和文件名一致。

上下文还是很大

检查：

- retain target 是否设置过高。
- 系统提示、工具定义、注入上下文是否额外占用大量 token。
- 是否重复注入历史摘要。
- 是否把大块工具输出保留下来了。

恢复到了错误 session

检查：

- active transcript 是否只靠 mtime 判断。
- last-good 是否过期。
- guardian 是否使用了不安全的 --continue。
- 是否有多个续航任务并发运行。

明明指定了模型，实际却变了

检查：

- 启动命令是否真的带了目标模型参数。
- transcript 里的 assistant model 字段是什么。
- 用户最近是否主动切过模型。
- Route Guard 是否记录了 drift line 和 anchor。
- 恢复后的新 transcript 是否仍包含漂移段。

Forge 后不会用工具

检查：

- 新 transcript 里是否还有 assistant tool_use 事件。
- 是否有匹配的 user tool_result.tool_use_id。
- 是否只保留了“我调用了工具”这类普通文本，而没有真实工具结构。
- 是否切点刚好切掉了最近所有工具回合。
- 工具结果是否被压缩成安全摘要，而不是被整段删除到只剩空上下文。

修复方向：

- 在裁剪后补 Tool Primer。
- 从旧 transcript 尾部向前找最近成功工具回合。
- 保留结构化 tool_use/tool_result，压缩结果正文。
- 在 resume 后跑一个无害的工具验证任务，确认会产生真实工具调用。

13. 简化伪代码

```
def reload_session(transcript_path, retain_tokens):
    events = read_jsonl(transcript_path)
    keepable = filter_keepable(events)
    cut = find_clean_boundary(keepable, retain_tokens)
    retained = keepable[cut:]
    retained = ensure_tool_primer(retained, keepable, min_rounds=1)

    new_sid = uuid4()
    rewritten = rewrite_session_chain(retained, new_sid)
    new_path = write_jsonl(new_sid, rewritten)

    stop_old_process()
    start_process(["claude", "--resume", new_sid])

    if verify_transcript_growth(new_path):
        mark_last_good(new_sid)
        archive_old(transcript_path)
        return True

    cleanup(new_path)
    recover_last_good()
    return False
```

这只是骨架。真实项目里还需要处理锁、超时、tmux、日志、用户提示和守护进程配合。

14. 最小可用版本

如果只做 MVP，建议先做这四件事：

1. 找当前 transcript。
2. 按行保留最后一段 JSONL。
3. 用新 session id 写成新文件。
4. claude --resume 后用 ping 验证。

等 MVP 跑稳，再加：

- 工具输出压缩。

- clean boundary。
- Tool Primer。
- last-good。
- guardian。
- 自动 watcher。
- Route Guard。
- 失败清理和归档。

15. 最重要的原则

这类系统不是为了“保存所有历史”，而是为了“保留继续完成任务所需的上下文”。

保留太少，会断片。

保留太多，会继续卡。

保留了脏东西，会把问题带进新 session。

好的 Swap/Forge 系统，本质是在“连续性”和“干净度”之间做工程取舍。